ĐẠI HỌC BÁCH KHOA

Khoa Khoa học và Kỹ thuật Máy tính

Báo cáo Bài tập E1–E3

Học sâu và ứng dụng trong thị giác máy tính

Mã môn học: CO5085

Giảng viên hướng dẫn: Lê Thành Sách, Nguyễn Quốc Minh

Sinh viên thực hiện: Nguyễn Trần Phước - MSSV: 2470576

Link GitHub Repo: https://github.com/NguyenTranPhuoc/CO5085_Assignment

Link GitHub Pages: https://nguyentranphuoc.github.io/CO5085_Assignment/

Thành phố Hồ Chí Minh, Năm 2026

Mục lục

1 Giới thiệu, Tập dữ liệu và Thiết lập thực nghiệm chung **2**
- 1.1 Tập dữ liệu Fashion-MNIST . . . 2
- 1.2 Cấu hình huấn luyện và Khả năng tái lập (Reproducibility) . . . 2

2 Bài tập E1 — Phân loại ảnh: Softmax, MLP, CNN **3**
- 2.1 Mục tiêu . . . 3
- 2.2 Kiến trúc chi tiết các mô hình (`E1/model.py`) . . . 3
  - 2.2.1 Mô hình 1: Softmax Classifier . . . 3
  - 2.2.2 Mô hình 2: Multi-Layer Perceptron (MLP Classifier) . . . 3
  - 2.2.3 Mô hình 3: Convolutional Neural Network (CNN Classifier) . . . 4
- 2.3 Kết quả định lượng và Phân tích so sánh E1 . . . 4
- 2.4 Phân tích đường cong học tập (Learning Curves - E1) . . . 5
- 2.5 Phân tích lỗi (Error Analysis): Confusion Matrix và Mẫu phân loại sai . . . 6

3 Bài tập E2 — Transformer: MSA và Tokenization **10**
- 3.1 Mục tiêu . . . 10
- 3.2 Thiết kế Kiến trúc Mô hình và Tokenization (`msa.py` và `E2/model.py`) . . . 10
  - 3.2.1 Hiện thực Multi-Head Self-Attention (MSA) và Transformer Encoder Block . . . 10
  - 3.2.2 Ba phương pháp lấy Token (Tokenization Strategies) . . . 11
- 3.3 Kết quả định lượng và Phân tích so sánh E2 . . . 11
- 3.4 Phân tích đường cong học (Learning Curves - E2) . . . 12
- 3.5 Trực quan hóa và Phân tích Bản đồ Chú ý (`[CLS]` Attention Maps) . . . 13

1 Giới thiệu, Tập dữ liệu và Thiết lập thực nghiệm chung

1.1 Tập dữ liệu Fashion-MNIST

Trong chuỗi bài tập thực hành từ E1 đến E3, nhóm lựa chọn tập dữ liệu **Fashion-MNIST** để đảm bảo tính nhất quán khi so sánh các họ kiến trúc mô hình khác nhau (Softmax, MLP, CNN ở E1; Vision Transformer với các chiến lược Tokenization ở E2; và LSTM/GRU ở E3).

- **Kích thước ảnh:** Ảnh đa cấp xám (grayscale) 1 kênh màu, độ phân giải 28×28 pixel.
- **Số lượng lớp:** 10 lớp đối tượng thời trang, bao gồm: `T-shirt/top` (0), `Trouser` (1), `Pullover` (2), `Dress` (3), `Coat` (4), `Sandal` (5), `Shirt` (6), `Sneaker` (7), `Bag` (8), và `Ankle boot` (9).
- **Phân chia dữ liệu (Data Split):** Tập dữ liệu gốc gồm $60,000$ ảnh huấn luyện và $10,000$ ảnh kiểm thử (test). Nhóm sử dụng tỉ lệ tách validation `val_ratio = 0.1` cố định với `seed = 42` thông qua `torch.utils.data.random_split`:
  - Tập huấn luyện (**Train set**): $54,000$ mẫu (844 batches với `batch_size = 64`).
  - Tập kiểm định (**Validation set**): $6,000$ mẫu.
  - Tập kiểm thử (**Test set**): $10,000$ mẫu ($1,000$ mẫu/lớp).
- **Tiền xử lý (Preprocessing):** Chuyển ảnh sang Tensor trong đoạn $[0, 1]$ và chuẩn hóa (Normalize) với $\mu = 0.5, \sigma = 0.5$, đưa giá trị điểm ảnh về đoạn $[-1, 1]$.



Hình 1: Một số mẫu ảnh đại diện từ tập dữ liệu Fashion-MNIST sau khi giải chuẩn hóa.

1.2 Cấu hình huấn luyện và Khả năng tái lập (Reproducibility)

Để đảm bảo tính công bằng khi đối sánh và khả năng tái lập kết quả (theo file `config_e1.json` và `config_e2.json`), quá trình huấn luyện của E1 và E2 dùng chung cách chia dữ liệu, hàm mất mát và thiết bị thực thi như Bảng 1:

Bảng 1: Cấu hình siêu tham số huấn luyện của Bài tập E1 và E2.

Thông số (Hyperparameter)	**Bài tập E1 (Softmax/MLP/CNN)**	**Bài tập E2 (Transformer/MSA)**
Random Seed	`42`	`42`
Batch size	64	64
Số vòng lặp (Epochs)	15	15
Hàm mất mát (Loss)	`CrossEntropyLoss`	`CrossEntropyLoss`
Bộ tối ưu (Optimizer)	Adam ($\text{lr} = 10^{-3}, \text{wd} = 10^{-4}$)	AdamW ($\text{lr} = 10^{-3}, \text{wd} = 10^{-2}$)
LR Scheduler	Constant LR	`CosineAnnealingLR` ($T_{\max} = 15$)
Gradient Clipping	Không	`clip_grad_norm_` ($\text{max_norm} = 1.0$)
Thiết bị (Device)	`cuda`	`cuda`
Chọn Checkpoint	Best Validation Accuracy	Best Validation Accuracy

2 Bài tập E1 — Phân loại ảnh: Softmax, MLP, CNN

2.1 Mục tiêu

Xây dựng vòng lặp huấn luyện PyTorch tường minh (`forward` → `loss` → `backward()` → `optimizer.step()`) và so sánh hiệu năng, dung lượng tham số (capacity), quá trình hội tụ cũng như các trường hợp dự đoán sai của 3 bộ phân loại:

1. **Softmax Classifier** (Linear probe trên vector điểm ảnh trải phẳng).
2. **Multi-Layer Perceptron (MLP)** nhiều tầng kết nối đầy đủ trên vector điểm ảnh trải phẳng.
3. **Convolutional Neural Network (CNN)** khai thác cấu trúc không gian 2 chiều của ảnh.

2.2 Kiến trúc chi tiết các mô hình (`E1/model.py`)

2.2.1 Mô hình 1: Softmax Classifier

Ảnh đầu vào $X \in \mathbb{R}^{B\times 1\times 28\times 28}$ được trải phẳng (flatten) thành vector $x \in \mathbb{R}^{B\times 784}$ và đi qua một tầng ánh xạ tuyến tính duy nhất:

$$\hat{y} = Wx + b, \quad W \in \mathbb{R}^{10\times 784},\ b \in \mathbb{R}^{10} \tag{1}$$

Tổng số tham số huấn luyện: $784 \times 10 + 10 = \mathbf{7,850}$ tham số.

2.2.2 Mô hình 2: Multi-Layer Perceptron (MLP Classifier)

Ảnh đầu vào sau khi trải phẳng thành vector 784 chiều được đưa qua mạng kết nối đầy đủ (Fully Connected) gồm 2 tầng ẩn kèm chuẩn hóa lô 1 chiều (`BatchNorm1d`), hàm kích hoạt phi tuyến `ReLU` và `Dropout` ($p = 0.2$):

- **Tầng ẩn 1:** `Linear`$(784 \to 256)$ → `BatchNorm1d`(256) → `ReLU` → `Dropout`(0.2).
- **Tầng ẩn 2:** `Linear`$(256 \to 128)$ → `BatchNorm1d`(128) → `ReLU` → `Dropout`(0.2).
- **Tầng đầu ra:** `Linear`$(128 \to 10)$.

Tổng số tham số huấn luyện:

$(784\times256+256)+(2\times256)+(256\times128+128)+(2\times128)+(128\times10+10) = \mathbf{235,914}$ tham số.

2.2.3 Mô hình 3: Convolutional Neural Network (CNN Classifier)

Thay vì trải phẳng ảnh ngay từ đầu làm mất cấu trúc lân cận không gian, CNN trích xuất đặc trưng qua 2 khối tích chập (Feature Extractor) trước khi đưa vào bộ phân loại (Classifier Head):

- **Khối tích chập 1:** `Conv2d`$(1 \to 32, \text{kernel } 3{\times}3, \text{padding } 1, \text{bias=False})$ $\to$ `BatchNorm2d`(32) $\to$ `ReLU` $\to$ `MaxPool2d`$(2 \times 2, \text{stride } 2)$. Kích thước đặc trưng giảm từ 28×28 xuống 14×14.
- **Khối tích chập 2:** `Conv2d`$(32 \to 64, \text{kernel } 3{\times}3, \text{padding } 1, \text{bias=False})$ $\to$ `BatchNorm2d`(64) $\to$ `ReLU` $\to$ `MaxPool2d`$(2 \times 2, \text{stride } 2)$. Kích thước đặc trưng giảm xuống $64 \times 7 \times 7$.
- **Khối phân loại (Head):** `Flatten`(3136) $\to$ `Linear`$(3136 \to 128)$ $\to$ `ReLU` $\to$ `Dropout`(0.25) $\to$ `Linear`$(128 \to 10)$.

Tổng số tham số huấn luyện: $\mathbf{421,738}$ tham số.

2.3 Kết quả định lượng và Phân tích so sánh E1

Bảng 2 tổng hợp kết quả thực nghiệm của 3 mô hình sau 15 epochs trên tập dữ liệu Fashion-MNIST (trích xuất từ `summary_results_e1.csv` và `training_log_e1.json`).

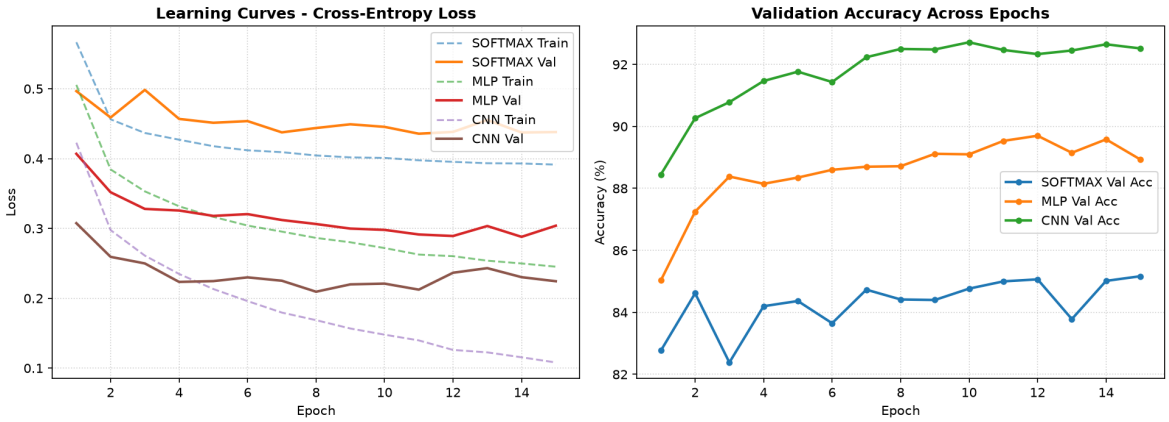
Bảng 2: So sánh kết quả thực nghiệm của 3 mô hình trong Bài tập E1.

Mô hình	Số tham số	Train Acc cuối	Best Val Acc	Test Acc	Test Loss	Thời gian (s)
SOFTMAX	7,850	86.33%	85.17% (Ep 15)	84.20%	0.4502	240.6
MLP	235,914	90.87%	89.70% (Ep 12)	89.04%	0.3044	251.8
CNN	**421,738**	**95.92%**	**92.72%** (Ep 10)	**91.98%**	**0.2507**	**245.7**

Nhận xét về Accuracy, Số tham số (Capacity) và Thời gian:

- **Từ Softmax lên MLP (+4.84% Test Acc):** Mô hình Softmax chỉ có 7,850 tham số và chỉ tạo được các siêu phẳng phân chia tuyến tính trong không gian 784 chiều, dẫn đến hiện tượng *underfitting* (Train Loss chững lại ở mức 0.3914 và Test Acc chỉ đạt 84.20%). Khi bổ sung các tầng ẩn phi tuyến, BatchNorm và Dropout, MLP tăng dung lượng lên 235,914 tham số và cải thiện mạnh độ chính xác trên tập Test lên 89.04%, đồng thời giảm Test Loss từ 0.4502 xuống 0.3044.
- **Từ MLP lên CNN (+2.94% Test Acc):** Mặc dù MLP có khả năng xấp xỉ phi tuyến tốt, việc flatten ảnh thành vector 1D khiến mô hình đánh mất hoàn toàn cấu trúc lân cận không gian 2D giữa các điểm ảnh. CNN với các bộ lọc tích chập 3×3 và MaxPool2d bảo toàn tính bất biến dịch chuyển cục bộ (local translation invariance), giúp trích xuất các đặc trưng hình thái (cạnh, cổ áo, tay áo, đế giày) hiệu quả vượt trội, đạt **Test Accuracy cao nhất là** 91.98% và **Test Loss thấp nhất là** 0.2507.
- **Thời gian huấn luyện:** Cả 3 mô hình có thời gian chạy 15 epochs trên GPU tương đương nhau (dao động 240.6s – 251.8s), bởi nút cổ chai thời gian chủ yếu nằm ở quá trình đọc/truyền dữ liệu (DataLoader I/O) đối với tập ảnh nhỏ 28×28.

2.4 Phân tích đường cong học tập (Learning Curves - E1)



Hình 2: Đường cong Cross-Entropy Loss (trái) và Validation Accuracy (phải) của 3 mô hình E1 qua 15 epochs.

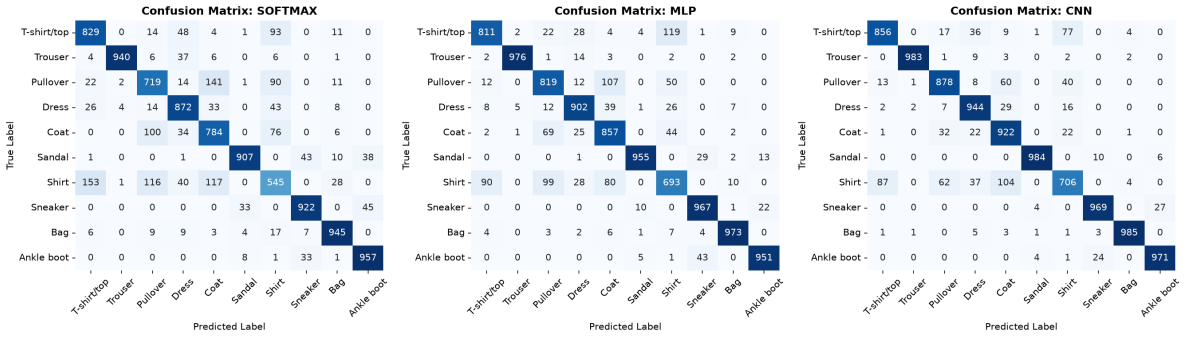
Quan sát Hình 2 và nhật ký huấn luyện (`training_log_e1.json`), ta rút ra các nhận định sau:

1. **Tốc độ hội tụ:** Ngay từ Epoch đầu tiên, CNN đã đạt Validation Accuracy 88.45%, cao hơn cả kết quả tốt nhất của Softmax sau 15 epochs (85.17%).

2. **Đặc tính khái quát hóa và Overfitting nhẹ ở cuối quá trình:**

   - Đối với **Softmax**, cả Train Loss (0.5666 $\rightarrow$ 0.3914) và Val Loss (0.4967 $\rightarrow$ 0.4380) giảm chậm và đi ngang sớm từ khoảng Epoch 7, minh chứng điển hình của mô hình có bias cao (high bias / underfitting).
   - Đối với **MLP**, mô hình hội tụ ổn định, đạt đỉnh Val Acc 89.70% tại Epoch 12 (Val Loss đạt cực tiểu 0.2880 tại Epoch 14). Nhờ có Dropout (0.2), khoảng cách giữa Train Acc (90.87%) và Val Acc (89.70%) rất nhỏ ($\sim$ 1.1%).
   - Đối với **CNN**, Train Loss giảm liên tục từ 0.4228 xuống 0.1081 (Train Acc đạt 95.92% ở Epoch 15). Trong khi đó, Val Loss đạt cực tiểu 0.2094 ở Epoch 8 và Val Acc đạt cực đại 92.72% ***ở* Epoch 10**, sau đó dao động nhẹ quanh vùng 92.3% – 92.6%. Việc lưu lại checkpoint tại Epoch có Val Acc tốt nhất (Epoch 10) đã giúp mô hình tránh được hiện tượng quá khớp (overfitting) ở các epoch cuối.

2.5 Phân tích lỗi (Error Analysis): Confusion Matrix và Mẫu phân loại sai



Hình 3: Ma trận nhầm lẫn (Confusion Matrix) trên tập Test (10,000 mẫu, 1,000 mẫu/lớp) của ba mô hình Softmax, MLP và CNN.

Từ các giá trị trên đường chéo chính và ngoài đường chéo của ba ma trận nhầm lẫn trong Hình 3, Bảng 3 thống kê số lượng mẫu phân loại đúng trên từng lớp (tương ứng với Recall của mỗi lớp do tập Test cân bằng tuyệt đối với 1,000 mẫu/lớp).

Bảng 3: Độ chính xác theo từng lớp (Per-class Accuracy / Recall) trích xuất từ Confusion Matrix.

Lớp đối tượng (Class)	**SOFTMAX**	**MLP**	**CNN**	**Tăng (CNN vs. Softmax)**
`T-shirt/top`	829 (82.9%)	811 (81.1%)	**856** (**85.6**%)	+2.7%
`Trouser`	940 (94.0%)	976 (97.6%)	**983** (**98.3**%)	+4.3%
`Pullover`	719 (71.9%)	819 (81.9%)	**878** (**87.8**%)	+15.9%
`Dress`	872 (87.2%)	902 (90.2%)	**944** (**94.4**%)	+7.2%
`Coat`	784 (78.4%)	857 (85.7%)	**922** (**92.2**%)	+13.8%
`Sandal`	907 (90.7%)	955 (95.5%)	**984** (**98.4**%)	+7.7%
`Shirt`	545 (54.5%)	693 (69.3%)	**706** (**70.6**%)	+16.1%
`Sneaker`	922 (92.2%)	967 (96.7%)	**969** (**96.9**%)	+4.7%
`Bag`	945 (94.5%)	973 (97.3%)	**985** (**98.5**%)	+4.0%
`Ankle boot`	957 (95.7%)	951 (95.1%)	**971** (**97.1**%)	+1.4%
Tổng số mẫu đúng / 10,000	8,420 (84.20%)	8,904 (89.04%)	**9,198** (**91.98**%)	**+778 mẫu** (**+7.78**%)

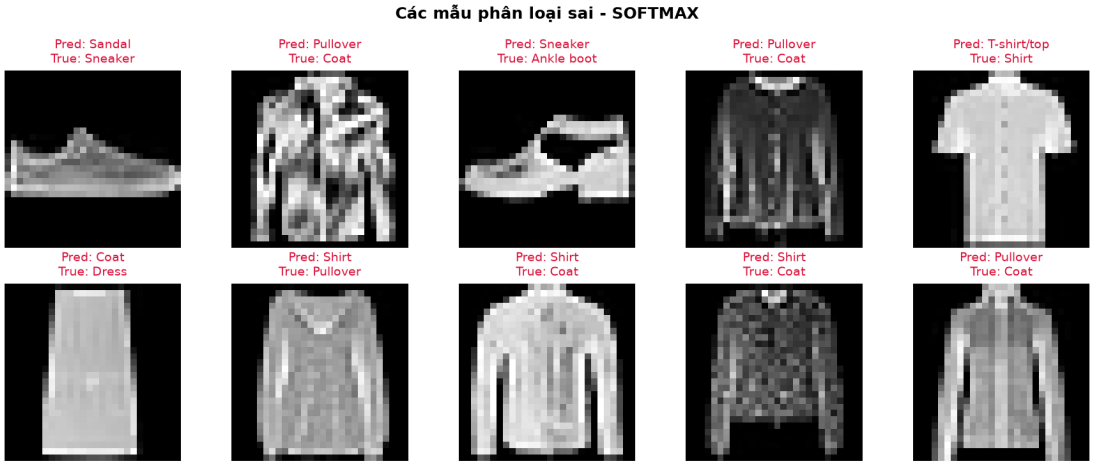
Phân tích định lượng từ Confusion Matrix (Hình 3 và Bảng 3):

1. **Nhóm lớp có hình thái đặc biệt, tách biệt rõ ràng** (`Bag`, `Sandal`, `Trouser`, `Ankle boot`, `Sneaker`):

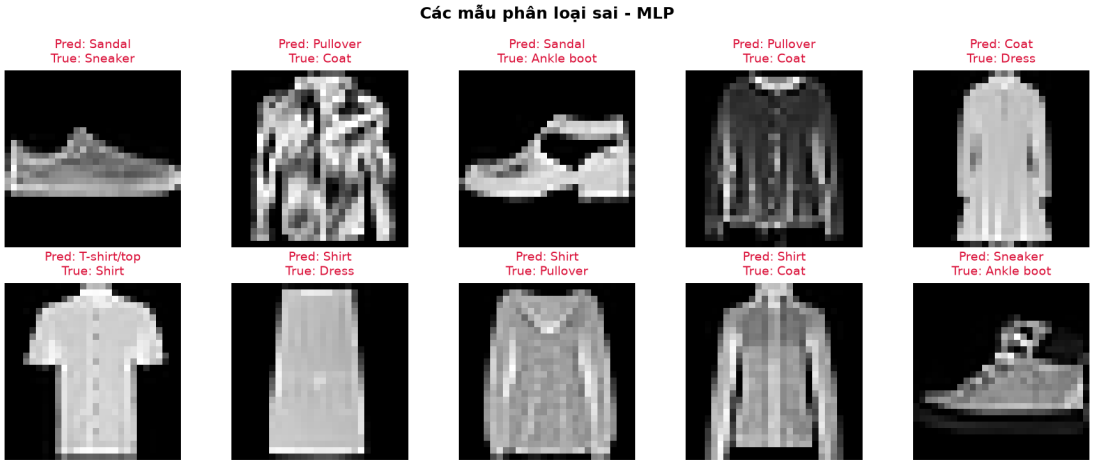
   - Đây là các lớp có độ chính xác rất cao ở cả 3 mô hình. Đặc biệt trên **CNN**, bốn lớp `Bag` (98.5%), `Sandal` (98.4%), `Trouser` (98.3%) và `Ankle boot` (97.1%) gần như không có sự nhầm lẫn chéo sang nhóm quần áo thân trên (các ô ngoài đường chéo giữa nhóm giày/túi và nhóm áo đều bằng 0 hoặc ≤ 5).
   - Trong nội bộ nhóm giày dép, **Softmax** nhầm lẫn khá nhiều giữa `Sandal` → `Sneaker` (43 mẫu), `Sandal` → `Ankle boot` (38 mẫu), và `Sneaker` → `Ankle boot` (45 mẫu). Khi chuyển sang **CNN**, nhờ khả năng nhận diện cấu trúc quai dép và cổ giày, lỗi `Sandal` → `Sneaker` giảm hơn 4 lần (từ 43 xuống còn 10 mẫu) và `Sandal` → `Ankle boot` giảm hơn 6 lần (từ 38 xuống chỉ còn 6 mẫu).

2. **Điểm nghẽn lớn nhất của tập dữ liệu — Cụm 4 lớp áo thân trên (`Shirt`, `T-shirt/top`, `Pullover`, `Coat`):**

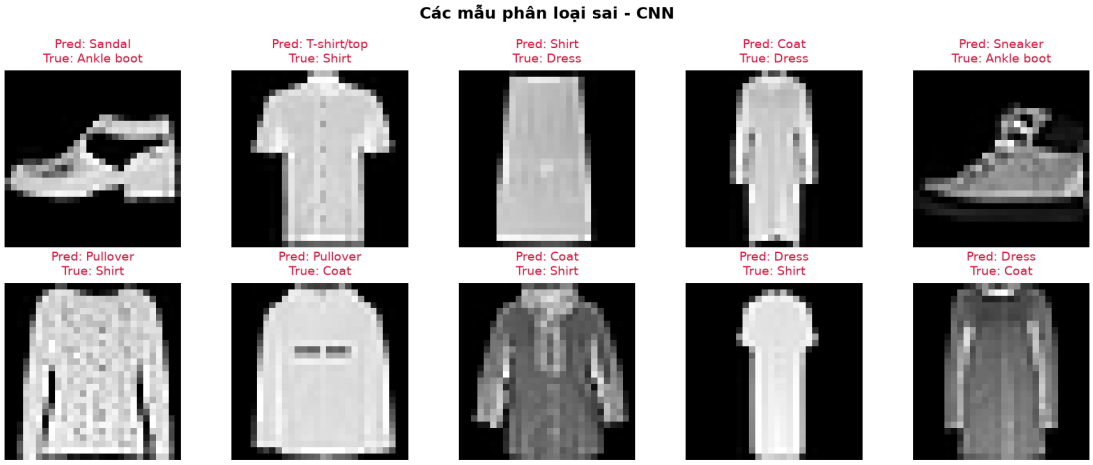
   - **Lớp `Shirt` (Áo sơ mi)** là lớp có độ chính xác thấp nhất trên cả 3 mô hình. Ở **Softmax**, chỉ có $545/1,000$ mẫu `Shirt` được dự đoán đúng (54.5%); có tới 153 mẫu bị nhầm thành `T-shirt/top`, 116 mẫu nhầm thành `Pullover`, 117 mẫu nhầm thành `Coat`, và 40 mẫu nhầm thành `Dress`. Nguyên nhân là vì áo sơ mi có cả biến thể ngắn tay (giống `T-shirt/top`) lẫn dài tay (giống `Pullover`/`Coat`), nên một bộ phân loại tuyến tính theo vị trí pixel không thể tách biệt được.
   - Khi sử dụng **MLP** và **CNN**, số mẫu đúng của lớp `Shirt` tăng vọt từ 545 lên 693 (MLP) và **706** (CNN, tăng +16.1%). Lỗi nhầm `Shirt` → `T-shirt/top` giảm từ 153 (Softmax) xuống 87 (CNN), và lỗi `Shirt` → `Pullover` giảm gần một nửa (từ 116 xuống 62 mẫu).
   - **Cặp nhầm lẫn hai chiều `Pullover` ↔ `Coat`:** Do đều là áo dài tay có phom dáng tương đồng trên ảnh xám 28×28, Softmax nhầm 141 mẫu `Pullover` thành `Coat` và 100 mẫu `Coat` thành `Pullover`. **CNN** đã giải quyết rất tốt cặp lỗi này nhờ trích xuất được đặc trưng đường khóa kéo/vạt áo trước ngực của `Coat`, giúp giảm lỗi `Pullover` → `Coat` từ 141 xuống 60 mẫu và lỗi `Coat` → `Pullover` từ 100 xuống chỉ còn 32 mẫu (đưa độ chính xác lớp `Pullover` tăng +15.9% và lớp `Coat` tăng +13.8%).



(a) Các mẫu phân loại sai của mô hình Softmax.



(b) Các mẫu phân loại sai của mô hình MLP.



(c) Các mẫu phân loại sai của mô hình CNN.

Hình 4: Trực quan hóa 10 mẫu phân loại sai đầu tiên trên tập Test của từng mô hình trong Bài E1.

Liên hệ với các mẫu ảnh phân loại sai thực tế (Hình 4):

- Các mẫu lỗi trực quan trong Hình 4 phản ánh chính xác các ô ngoài đường chéo

có giá trị lớn trên Confusion Matrix: phần lớn các mẫu sai rơi vào cụm `Shirt` ↔ `T-shirt/top` (mẫu số 5 của Softmax, mẫu số 6 của MLP, mẫu số 2 của CNN), `Coat` ↔ `Pullover/Shirt` (mẫu số 2, 4, 9 của Softmax/MLP; mẫu số 6, 7, 8, 9 của CNN) và `Dress` ↔ `Coat/Shirt` (mẫu số 8, 10 của Softmax; mẫu số 10 của CNN).

- Ở nhóm giày dép, mẫu số 1 của Softmax và MLP là một chiếc `Sneaker` cổ thấp bị nhầm thành `Sandal`, nhưng **CNN đã dự đoán đúng mẫu này**. Tuy nhiên, những mẫu có độ nhập nhằng thị giác cao như chiếc `Ankle boot` khoét lỗ lớn ở thân giống hệt quai dép `Sandal` (mẫu số 3 của Softmax/MLP và mẫu số 1 của CNN) vẫn bị cả 3 mô hình dự đoán sai.

3 Bài tập E2 — Transformer: MSA và Tokenization

3.1 Mục tiêu

Hiện thực cơ chế **Multi-Head Self-Attention (MSA)** từ các phép toán cơ bản (from scratch) và tích hợp vào kiến trúc Vision Transformer (ViT) nông để phân loại ảnh trên tập dữ liệu **Fashion-MNIST**; đồng thời khảo sát ảnh hưởng của 3 chiến lược trích xuất token (`PATCH`, `PIXEL`, `CNN_STEM`) đến độ chính xác, chi phí tính toán và bản đồ chú ý (Attention Map).

3.2 Thiết kế Kiến trúc Mô hình và Tokenization (`msa.py` và `E2/model.py`)

3.2.1 Hiện thực Multi-Head Self-Attention (MSA) và Transformer Encoder Block

Trong tệp `msa.py`, lớp `MultiHeadSelfAttention` được hiện thực hoàn toàn từ các phép toán ma trận cơ bản mà không phụ thuộc vào `nn.MultiheadAttention`:

- Với đầu vào $X \in \mathbb{R}^{B\times n\times d_{\text{model}}}$ ($d_{\text{model}} = 64$, số lượng head $H = 4 \Rightarrow d_k = d_{\text{model}}/H = 16$), các tensor truy vấn ($Q$), khóa ($K$), giá trị ($V$) được tính qua 3 tầng tuyến tính $W_q, W_k, W_v \in \mathbb{R}^{64\times 64}$ và tách thành H đầu chú ý có kích thước (B, H, n, d_k):

$$Q = \text{reshape}(XW_q), \quad K = \text{reshape}(XW_k), \quad V = \text{reshape}(XW_v) \tag{2}$$

- Điểm số chú ý (attention scores) được tính bằng tích vô hướng có chia tỉ lệ $\sqrt{d_k}$, chuẩn hóa qua hàm `softmax`, áp dụng `Dropout` ($p = 0.1$) và nhân với V:

$$\text{Attn}(Q, K, V) = \text{Dropout}\left(\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)\right)V \tag{3}$$

- Đầu ra của H heads được chuyển vị, ghép nối (concatenate) lại về kích thước (B, n, d_{model}) và chiếu qua tầng tuyến tính $W_o \in \mathbb{R}^{64\times 64}$.

- **TransformerEncoderBlock (Pre-LayerNorm):** Mỗi khối encoder áp dụng cấu trúc chuẩn hóa trước (*Pre-Norm*) giúp ổn định đạo hàm khi huấn luyện:

$$X' = X + \text{MSA}(\text{LayerNorm}_1(X)) \tag{4}$$
$$Y = X' + \text{FFN}(\text{LayerNorm}_2(X')) \tag{5}$$

  trong đó khối FFN gồm `Linear`$(64 \to 256) \to$ `GELU` $\to$ `Dropout`$(0.1) \to$ `Linear`$(256 \to 64) \to$ `Dropout`(0.1). Toàn bộ mạng `VisionTransformerClassifier` xếp chồng $L = 2$ khối `TransformerEncoderBlock`.

- **Đối chiếu với MSA của PyTorch:** Khi đối chiếu với module dựng sẵn `nn.MultiheadAttention(num_heads=4, batch_first=True)`, bản tự viết có cùng số lượng tham số trong mỗi tầng MSA ($4 \times (64 \times 64 + 64) = 16,640$ tham số), cho đầu ra tương đương về mặt toán học nhưng cho phép trích xuất trực tiếp ma trận trọng số chú ý attn_weights $\in \mathbb{R}^{B\times H\times n\times n}$ phục vụ trực quan hóa.

3.2.2 Ba phương pháp lấy Token (Tokenization Strategies)

Trước khi đi vào chuỗi các khối Transformer Encoder, ảnh đầu vào $X \in \mathbb{R}^{B\times1\times28\times28}$ được chuyển đổi thành chuỗi N token đặc trưng $d_{\text{model}} = 64$ chiều, sau đó ghép thêm 1 token phân loại `[CLS]` ở vị trí đầu tiên (tổng độ dài chuỗi $n = N + 1$) và cộng với **Learnable Positional Embedding** $P \in \mathbb{R}^{1\times(N+1)\times64}$:

1. **Patch Tokenizer (`PATCH`):** Chia ảnh 28×28 thành lưới các ô vuông không giao nhau kích thước 4×4 thông qua tầng tích chập `Conv2d`$(1 \to 64, \text{kernel } 4\times4, \text{stride } 4)$.
   - Số token không gian: $N = (28/4)^2 = 49$ tokens.
   - Tổng độ dài chuỗi sau khi ghép `[CLS]`: $n = 49 + 1 = \mathbf{50}$ tokens.
   - Tổng số tham số toàn mô hình: $\mathbf{105,098}$ tham số.
2. **Pixel Tokenizer (`PIXEL`):** Giảm chiều ảnh từ 28×28 xuống 14×14 bằng `AvgPool2d`$(2\times2, \text{stride } 2)$ để tránh bùng nổ bậc hai của ma trận chú ý (784^2), sau đó xem mỗi điểm ảnh (pixel) là một token độc lập và chiếu tuyến tính từ 1 kênh lên 64 kênh qua `Linear`$(1 \to 64)$.
   - Số token không gian: $N = 14 \times 14 = 196$ tokens.
   - Tổng độ dài chuỗi sau khi ghép `[CLS]`: $n = 196 + 1 = \mathbf{197}$ tokens.
   - Tổng số tham số toàn mô hình: $\mathbf{113,546}$ tham số (tăng thêm chủ yếu do bảng Positional Embedding dài 197×64).
3. **CNN-Stem Tokenizer (`CNN_STEM`):** Đưa ảnh qua một nhánh tích chập nông gồm 2 tầng giảm kích thước liên tiếp:

   `Conv2d`$(1 \to 32, k = 3, s = 2, p = 1) \to$ `BN` $\to$ `GELU` $\to$ `Conv2d`$(32 \to 64, k = 3, s = 2, p = 1) \to$ `BN` $\to$

   Feature map đầu ra có kích thước $64 \times 7 \times 7$, mỗi vị trí không gian trên lưới 7×7 được xem là một token.
   - Số token không gian: $N = 7 \times 7 = 49$ tokens.
   - Tổng độ dài chuỗi sau khi ghép `[CLS]`: $n = 49 + 1 = \mathbf{50}$ tokens.
   - Tổng số tham số toàn mô hình: $\mathbf{122,922}$ tham số.

3.3 Kết quả định lượng và Phân tích so sánh E2

Bảng 4 trình bày kết quả thực nghiệm của 3 phương pháp Tokenization sau 15 epochs huấn luyện với AdamW và CosineAnnealingLR (trích xuất từ `summary_results_e2.csv` và `training_log_e2.json`).

Bảng 4: So sánh hiệu năng của 3 phương pháp Tokenization trong Bài tập E2.

Tokenizer	Chuỗi (n)	Số tham số	Train Acc cuối	Best Val Acc	Test Acc	Test Loss	Thời gian (s)
PATCH (4×4)	50	105,098	89.29%	88.55% (Ep 15)	88.30%	0.3278	317.1
PIXEL (14×14)	197	113,546	86.10%	86.20% (Ep 15)	84.72%	0.3998	315.7
CNN_STEM	**50**	**122,922**	**92.59%**	**90.87%** (Ep 15)	**90.33%**	**0.2828**	**287.4**

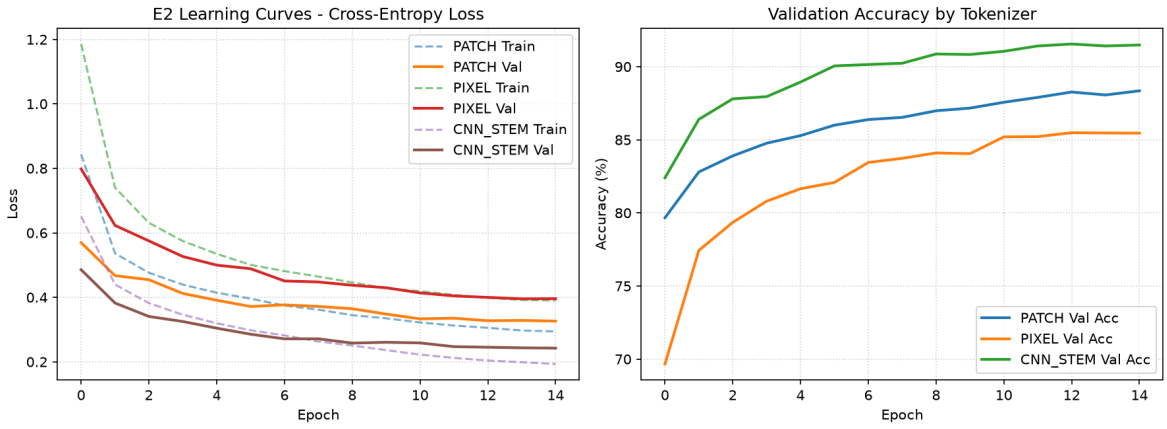
Nhận xét chi tiết về Accuracy, Độ dài chuỗi và Thời gian thực thi:

1. **Hiệu quả vượt trội của `CNN_STEM` (90.33% Test Acc):** So với `PATCH` (88.30%) và `PIXEL` (84.72%), `CNN_STEM` cải thiện lần lượt +2.03% và +5.61% độ chính xác trên tập Test, đồng thời giảm mạnh Test Loss xuống 0.2828. Nguyên nhân cốt lõi là các tầng tích chập có chồng lấp (3×3, stride 2, padding 1) ở phần Stem đã đưa thiên kiến quy nạp không gian cục bộ (local spatial inductive bias) vào mô hình, giúp mỗi token trong lưới 7×7 mang thông tin cạnh và kết cấu vùng lân cận phong phú hơn hẳn việc cắt lát rời rạc không chồng lấp của `PATCH`.

2. **Hạn chế của `PIXEL` Tokenizer (84.72% Test Acc):** Mặc dù có độ dài chuỗi lớn gấp gần 4 lần ($n = 197$ so với $n = 50$, khiến kích thước ma trận Self-Attention tăng từ $50^2 = 2,500$ lên $197^2 = 38,809$ phần tử trên mỗi head), `PIXEL` lại có độ chính xác thấp nhất (84.72%, chỉ nhỉnh hơn Softmax ở E1 một chút). Điều này xảy ra vì mỗi token đầu vào chỉ là một giá trị cường độ sáng vô hướng đơn lẻ ($\mathbb{R}^1 \to \mathbb{R}^{64}$), thiếu hoàn toàn thông tin ngữ cảnh hình học cục bộ, khiến cơ chế Self-Attention rất khó phân biệt các điểm ảnh có cùng độ sáng nhưng thuộc các bộ phận khác nhau của trang phục.

3. **Đối sánh giữa ViT (E2) và CNN thuần túy (E1):** Mô hình tốt nhất ở E2 là `CNN_STEM` đạt 90.33% Test Acc với chỉ **122,922** tham số (ít hơn 3.4 lần so với 421,738 tham số của CNN ở E1 đạt 91.98%), và vượt qua MLP ở E1 (89.04% với 235,914 tham số). Trong khi đó, ViT thuần túy (`PATCH`) đạt 88.30%, thấp hơn CNN truyền thống do Transformer đòi hỏi lượng dữ liệu huấn luyện lớn hơn nhiều để tự học các quy luật không gian từ đầu.

3.4 Phân tích đường cong học (Learning Curves - E2)



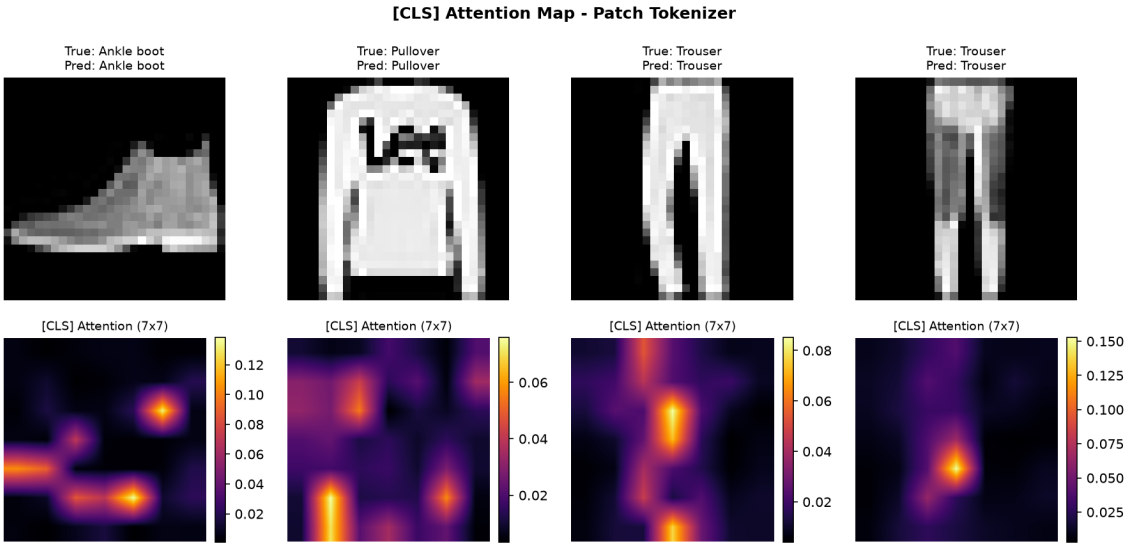
Hình 5: Đường cong Cross-Entropy Loss (trái) và Validation Accuracy (phải) của 3 phương pháp Tokenization qua 15 epochs.

Quan sát Hình 5 và dữ liệu từ `training_log_e2.json`:

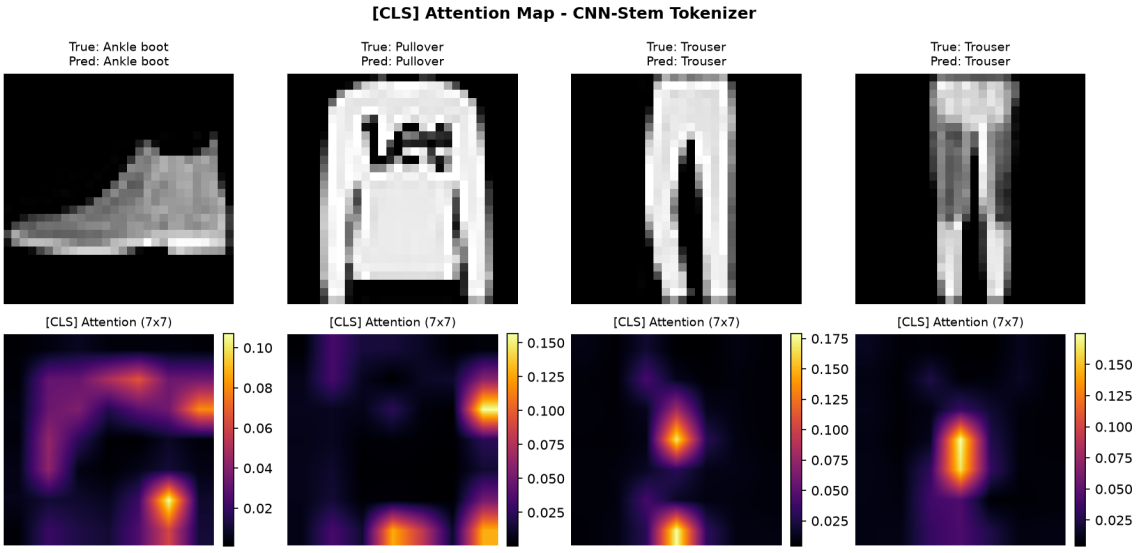
- **Tốc độ hội tụ và khoảng cách hiệu năng nhất quán:** Ngay từ Epoch 1, `CNN_STEM` đã đạt Val Acc 82.98% (so với 80.92% của `PATCH` và 71.52% của `PIXEL`). Khoảng cách phân cấp `CNN_STEM` > `PATCH` > `PIXEL` được duy trì xuyên suốt toàn bộ 15 epochs.

- **Ảnh hưởng của CosineAnnealingLR và AdamW:** Nhờ bộ điều chỉnh tốc độ học giảm dần theo hàm Cosine và kỹ thuật cắt chuẩn đạo hàm (`clip_grad_norm_ = 1.0`), cả 3 mô hình Transformer đều hội tụ cực kỳ mượt mà, không xuất hiện các gai nhọn dao động (spikes) trên đường Val Loss và tất cả đều đạt đỉnh Validation Accuracy tại Epoch cuối cùng (Epoch 15).

3.5 Trực quan hóa và Phân tích Bản đồ Chú ý (`[CLS]` Attention Maps)



(a) Bản đồ chú ý `[CLS]` Attention Map (7×7) với **Patch Tokenizer**.



(b) Bản đồ chú ý `[CLS]` Attention Map (7×7) với **CNN-Stem Tokenizer**.

Hình 6: So sánh trọng số chú ý từ token `[CLS]` tới $7 \times 7 = 49$ vùng không gian trên 4 mẫu thử nghiệm (`Ankle boot`, `Pullover`, `Trouser`, `Trouser`).

Hình 6 minh họa trọng số chú ý trung bình giữa các head từ token đại diện [CLS] tới 49 token không gian (7×7) ở tầng Transformer cuối cùng. Qua đối chiếu trực quan giữa Hình 6a (PATCH) và Hình 6b (CNN_STEM), ta nhận thấy:

1. **Trên mẫu Ankle boot (cột 1):** PATCH tập trung vào một vài ô rời rạc ở mũi giày, gót giày và cổ giày. Ngược lại, CNN_STEM tạo ra vùng chú ý bao phủ liên tục dọc theo đường viền cổ giày, mu bàn chân và phần gót đế giày — đây chính là các vùng đặc trưng quyết định để phân biệt Ankle boot với Sneaker hay Sandal.

2. **Trên mẫu Pullover có chữ "Lee" (cột 2):** CNN_STEM tập trung trọng số chú ý rất rõ vào phần ống tay áo dài hai bên và gấu áo dưới (vùng biên phân biệt áo len dài tay với áo phông), trong khi PATCH bị phân tán một phần sự chú ý vào vùng nền và họa tiết chữ ở ngực áo.

3. **Trên hai mẫu Trouser (cột 3 và cột 4):** Cả hai mô hình đều học được cách chú ý mạnh nhất vào **vùng khe hở tách giữa hai ống quần** và phần ống quần dưới. Đặc biệt, bản đồ chú ý của CNN_STEM có độ tương phản cao và định vị tâm điểm tách hai ống quần vô cùng sắc nét (trọng số đạt tới $0.16 - 0.18$), giải thích vì sao mô hình phân loại lớp Trouser đạt độ chính xác gần như tuyệt đối.